

Program

```
#include <stdio.h>
#include <stdlib.h> // For exit()

#define MAX_SIZE 5 // Defines the maximum size of the stack

// Structure to represent the stack
struct Stack {
    int items[MAX_SIZE];
    int top;
};

//--- Function Prototypes ---
void initialize(struct Stack *s);
int isFull(struct Stack *s);
int isEmpty(struct Stack *s);
void push(struct Stack *s, int value);
int pop(struct Stack *s);
int peek(struct Stack *s);
void display(struct Stack *s);

//--- Main Program Logic ---
int main() {
    struct Stack s;
    initialize(&s);
    int option, value;

    while (1) {
        printf("\n--- Stack Menu ---\n");
        printf("1. Push\n");
        printf("2. Pop\n");
        printf("3. Peek (View Top)\n");
        printf("4. Display Stack\n");
        printf("5. Exit\n");
        printf("Choose an option: ");
        scanf("%d", &option);

        switch (option) {
            case 1:
                printf("Enter value to push: ");
```

```

        scanf("%d", &value);
        push(&s, value);
        break;

    case 2:
        value = pop(&s);
        if (value != -1) { // -1 indicates underflow in our implementation
            printf("Popped value: %d\n", value);
        }
        break;

    case 3:
        value = peek(&s);
        if (value != -1) {
            printf("Top element is: %d\n", value);
        }
        break;

    case 4:
        display(&s);
        break;

    case 5:
        printf("Exiting program. Goodbye!\n");
        exit(0);

    default:
        printf("Invalid option! Please choose between 1 and 5.\n");
    }
}

return 0;
}

//--- Function Definitions ---

/**
 * @brief Initializes the stack by setting top to -1.
 */
void initialize(struct Stack *s) {
    s->top = -1;
}

```

```

/**
 * @brief Checks if the stack is full.
 * @return 1 if full, 0 otherwise.
 */
int isFull(struct Stack *s) {
    return s->top == MAX_SIZE - 1;
}

/**
 * @brief Checks if the stack is empty.
 * @return 1 if empty, 0 otherwise.
 */
int isEmpty(struct Stack *s) {
    return s->top == -1;
}

/**
 * @brief Adds an element to the top of the stack.
 */
void push(struct Stack *s, int value) {
    if (isFull(s)) {
        printf("Stack Overflow! Cannot push element.\n");
    } else {
        s->top++;
        s->items[s->top] = value;
        printf("%d pushed to stack.\n", value);
    }
}

/**
 * @brief Removes and returns the top element of the stack.
 * @return The popped element, or -1 if the stack is empty.
 */
int pop(struct Stack *s) {
    if (isEmpty(s)) {
        printf("Stack Underflow! Cannot pop from an empty stack.\n");
        return -1; // Return a sentinel value indicating error
    } else {
        int poppedValue = s->items[s->top];
        s->top--;
        return poppedValue;
    }
}

```

```

/**
 * @brief Returns the top element of the stack without removing it.
 * @return The top element, or -1 if the stack is empty.
 */
int peek(struct Stack *s) {
    if (isEmpty(s)) {
        printf("Stack is empty! Nothing to peek.\n");
        return -1; // Return a sentinel value indicating error
    } else {
        return s->items[s->top];
    }
}

/**
 * @brief Displays all the elements in the stack from top to bottom.
 */
void display(struct Stack *s) {
    if (isEmpty(s)) {
        printf("Stack is empty.\n");
    } else {
        printf("Stack elements (from top to bottom):\n");
        for (int i = s->top; i >= 0; i--) {
            printf("%d\n", s->items[i]);
        }
    }
}

```

Sample Execution Output

```

--- Stack Menu ---
1. Push
2. Pop
3. Peek (View Top)
4. Display Stack
5. Exit
Choose an option: 1
Enter value to push: 10
10 pushed to stack.

--- Stack Menu ---

```

1. Push
2. Pop
3. Peek (View Top)
4. Display Stack
5. Exit

Choose an option: 1

Enter value to push: 20

20 pushed to stack.

--- Stack Menu ---

1. Push
2. Pop
3. Peek (View Top)
4. Display Stack
5. Exit

Choose an option: 1

Enter value to push: 30

30 pushed to stack.

--- Stack Menu ---

1. Push
2. Pop
3. Peek (View Top)
4. Display Stack
5. Exit

Choose an option: 4

Stack elements (from top to bottom):

30

20

10

--- Stack Menu ---

1. Push
2. Pop
3. Peek (View Top)
4. Display Stack
5. Exit

Choose an option: 3

Top element is: 30

--- Stack Menu ---

1. Push
2. Pop

3. Peek (View Top)
4. Display Stack
5. Exit
Choose an option: 2
Popped value: 30

--- Stack Menu ---

1. Push
2. Pop
3. Peek (View Top)
4. Display Stack
5. Exit
Choose an option: 4
Stack elements (from top to bottom):
20
10

--- Stack Menu ---

1. Push
2. Pop
3. Peek (View Top)
4. Display Stack
5. Exit
Choose an option: 2
Popped value: 20

--- Stack Menu ---

1. Push
2. Pop
3. Peek (View Top)
4. Display Stack
5. Exit
Choose an option: 2
Popped value: 10

--- Stack Menu ---

1. Push
2. Pop
3. Peek (View Top)
4. Display Stack
5. Exit
Choose an option: 2
Stack Underflow! Cannot pop from an empty stack.

--- Stack Menu ---

1. Push
2. Pop
3. Peek (View Top)
4. Display Stack
5. Exit

Choose an option: 5

Exiting program. Goodbye!

Algorithms and Analysis

Overall Program Algorithm

This describes the flow of the main application.

1. **Start** the program.
2. **Initialize Stack**: Create a Stack structure and initialize its top index to -1.
3. **Start Main Loop**: Begin an infinite while loop to display the menu.
4. **Display Menu**: Show the user the options: 1 for Push, 2 for Pop, 3 for Peek, 4 for Display, and 5 to Exit.
5. **Get User Choice**: Read the integer option chosen by the user.
6. **Process Choice (Switch Statement)**:
 - **Case 1 (Push)**: Prompt the user for a value, then call the push function with that value.
 - **Case 2 (Pop)**: Call the pop function. If it doesn't return an error, print the popped value.
 - **Case 3 (Peek)**: Call the peek function. If it doesn't return an error, print the top value.
 - **Case 4 (Display)**: Call the display function to print all stack elements.
 - **Case 5 (Exit)**: Print an exit message and terminate the program using `exit(0)`.
 - **Default**: If the choice is invalid, print an error message.
7. **Loop Continuation**: The loop automatically repeats from Step 4.

Push Operation

Algorithm:

Adds an element to the top of the stack.

1. Check if the stack is full (i.e., if top is equal to `MAX_SIZE - 1`).
2. If the stack is full, print a "Stack Overflow" error and terminate the operation.
3. If the stack is not full, increment top by 1.
4. Insert the new element at the array index indicated by the new top.

Complexity Analysis:

- **Time Complexity**: $O(1)$. The operation takes a constant amount of time, as it only involves a single check, an increment, and an array assignment.
- **Space Complexity**: $O(1)$. No extra space is required.

Pop Operation

Algorithm:

Removes and returns the element from the top of the stack.

1. Check if the stack is empty (i.e., if top is equal to -1).
2. If the stack is empty, print a "Stack Underflow" error and return an error code.

3. If the stack is not empty, retrieve the element at the array index top.
4. Decrement top by 1.
5. Return the retrieved element.

Complexity Analysis:

- **Time Complexity:** $O(1)$. This is a constant time operation, involving a check, a decrement, and returning a value.
- **Space Complexity:** $O(1)$. No extra space is required.

Peek (Top) Operation

Algorithm:

Returns the element from the top of the stack without removing it.

1. Check if the stack is empty (i.e., if top is equal to -1).
2. If the stack is empty, print an error message and return an error code.
3. If the stack is not empty, return the element at the array index top.

Complexity Analysis:

- **Time Complexity:** $O(1)$. This operation simply returns a value from an array index.
- **Space Complexity:** $O(1)$. No extra space is required.